```
a = "The moon is made of cheese"
show_regexp(a, /\w+/)           # => ->The<- moon is made of cheese
show_regexp(a, /\s.*\s/)        # => The-> moon is made of <-cheese
show_regexp(a, /\s.*?\s/)       # => The-> moon <-is made of cheese
show_regexp(a, /[aeiou]{2,99}/) # => The m->oo<-n is made of cheese
show_regexp(a, /mo?o/)          # => The ->moo<-n is made of cheese
show_regexp(a, /mo??o/)         # => The ->mo<-on is made of cheese
```

The lazy versions, on line three and six behave differently than their matching greedy versions on lines two and five.

Be very careful when using the * modifier. It matches zero or more occurrences. We often forget about the zero part. In particular, a pattern that contains just a * repetition will always match, whatever string you pass it. For example, the pattern /a*/ will always match, because every string contains zero or more a's.

Both of these examples match an empty substring at the start of the string…

```
a = "The moon is made of cheese"
show_regexp(a, /m*/) # => -><-The moon is made of cheese
show_regexp(a, /Z*/) # => -><-The moon is made of cheese
```

Alternation

We know that the vertical bar is special, because our line-splitting pattern had to escape it with a backslash. That's because an unescaped vertical bar, as in |, matches either the construct that precedes it or the construct that follows it:

```
a = "red ball blue sky"
show_regexp(a, /d|e/)               # => r->e<-d ball blue sky
show_regexp(a, /al|lu/)             # => red b->al<-l blue sky
show_regexp(a, /red ball|angry sky/) # => ->red ball<- blue sky
```

There's a trap for the unwary here, because | has a very low precedence. The last example in the previous lines matches *red ball* or *angry sky*, not *red ball sky* or *red angry sky*. To match *red ball sky* or *red angry sky*, you'd need to override the default precedence using grouping, /red (ball|angry) sky/.

Grouping

You can use parentheses to group terms within a regular expression. Everything within the group is treated as a single regular expression.

The first example here, without a group, matches an 'a' followed by one or more 'n's. The second, using a group, matches the sequence 'an' one or more times.

```
show_regexp('banana', /an+/)   # => b->an<-ana
show_regexp('banana', /(an)+/) # => b->anan<-a
```

```
a = 'red ball blue sky'
show_regexp(a, /blue|red/)             # => ->red<- ball blue sky
show_regexp(a, /(blue|red) \w+/)       # => ->red ball<- blue sky
show_regexp(a, /(red|blue) \w+/)       # => ->red ball<- blue sky
show_regexp(a, /red|blue \w+/)         # => ->red<- ball blue sky
show_regexp(a, /red (ball|angry) sky/) # => no match
a = 'the red angry sky'
```